

Check a box and save the file (Ctrl/Cmd+S) to keep your progress.

JAVA CORE

- ☐ Multithreading: synchronized vs volatile vs ReentrantLock
- ☐ ExecutorService, thread pools, CompletableFuture
- ☐ ConcurrentHashMap internals vs HashMap
- ☐ Deadlock / livelock / race condition (with examples)
- ☐ JMM: heap vs stack, GC basics (G1GC)
- ☐ Reference types: Strong / Weak / Soft / Phantom
- ☐ HashMap internal working (hashCode, equals, resize, collision)
- ☐ ArrayList vs LinkedList vs ArrayDeque
- ☐ TreeMap vs LinkedHashMap
- ☐ Java 8+: Streams, Optional, functional interfaces
- ☐ default / static methods in interfaces
- ☐ CompletableFuture chaining

SPRING CORE

- ☐ IoC & DI: constructor vs field injection (why constructor)
- ☐ Bean lifecycle & scopes (singleton, prototype, etc.)
- ☐ @Component vs @Bean vs @Service
- ☐ @SpringBootApplication - what it does under the hood
- ☐ Auto-configuration mechanism (spring.factories)
- ☐ Embedded server, application.properties vs yaml

SPRING DATA JPA

- ☐ @Entity, @Transactional - isolation levels & propagation
- ☐ REQUIRES_NEW and other propagation types (fintech favorite)

Check a box and save the file (Ctrl/Cmd+S) to keep your progress.

- ☐ N+1 problem and fixes (fetch joins, @EntityGraph)
- ☐ Lazy vs Eager loading

SPRING SECURITY

- ☐ JWT auth flow end to end
- ☐ Filter chain, OncePerRequestFilter
- ☐ OAuth2 basics

REST & EXCEPTION HANDLING

- ☐ @ControllerAdvice, @ExceptionHandler
- ☐ ResponseEntity, proper HTTP status codes
- ☐ Validation: @Valid, @NotNull, custom validators

MICROSERVICES (SDE2 MUST-KNOW)

- ☐ Circuit breaker pattern (Resilience4j)
- ☐ Service discovery (Eureka) + API Gateway
- ☐ Distributed tracing (Sleuth/Zipkin)
- ☐ Saga pattern
- ☐ Idempotency in APIs

YOUR PERSONAL HIGH-ROI TOPICS

- ☐ Caffeine Cache internals & eviction policies (articulate your project)
- ☐ Kafka: consumer groups, partition rebalancing
- ☐ Kafka: at-least-once vs exactly-once, offset management

Check a box and save the file (Ctrl/Cmd+S) to keep your progress.

- ☐ @Transactional deep dive - REQUIRES_NEW walkthrough

DESIGN PATTERNS

- ☐ Strategy Pattern - implement live (you've used this already)
- ☐ Factory Pattern
- ☐ Singleton Pattern (thread-safe variants)
- ☐ Builder Pattern
- ☐ Observer Pattern

TESTING

- ☐ JUnit 5 basics: annotations, assertions, lifecycle
- ☐ Mockito: @Mock, @InjectMocks, when/thenReturn
- ☐ @MockBean vs @Mock
- ☐ @SpringBootTest vs @WebMvcTest vs @DataJpaTest
- ☐ How to unit test a service with a repository dependency

EXCEPTION HANDLING NUANCES

- ☐ Checked vs unchecked exceptions
- ☐ try-with-resources
- ☐ Custom exception hierarchy design

GENERICICS

- ☐ Bounded type parameters
- ☐ Wildcards: ? extends vs ? super

Check a box and save the file (Ctrl/Cmd+S) to keep your progress.

SPRING AOP

- ☐ @Aspect, @Before / @Around / @After
- ☐ Pointcut expressions
- ☐ AOP use case: logging framework (ties to your Vimaan work)